* Note :- As a developer we are responsible to handle the Exception. System admin is responsible to handle the error because we cannot recover from error. It represents **serious problem** we cannot recover.

* A developer is responsible for every type of exception either checked OR un-checked.

RMP to show that **java.lang.Exception** is the super class for all types of exception in java :

```
package com.rev1.basic;

public class ExceptionException {

    public static void main(String[] args)
    {
        Exception e = new ArithmeticException();
        IO.println(e);  //java.lang.ArithmeticException

        Exception ex = new NullPointerException("ur m is error message");
        IO.println(ex.getMessage());  //java.lang.NullPointerException : ur m is error message
    }

}
```

Exception Format:

The java software people has provided the **format of exception** so whenever we print **exception object** by using toString() method then the format is :

Fully Qualified Name : errorMessage

Package Name + Class Name : errorMessage

This logic is written in the toString() method of Throwable class.

MAIN that describes, whenever an exception is encountered in the program then our program will be terminated abnormally (halt in the middle)

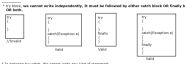
```
package com.rev1.basic;

public class AbnormalTermination
{
    public static void main(String[] args)
    {
        IO.println("Main method started");
        int i = 20;
        int j = 30;
        int k = i + j;
        IO.println("Result value is: "+k);
        IO.println("Main method ended");
    }

}
```

Note :
In the above program, due to 20/0; program will be terminated in the middle, IT IS CALLED **ABNORMAL TERMINATION**.
Actually JVM has a **default exception handler** which is responsible to print the exception details and terminate the program abnormally. This is not recommended.

In order to handle the exception and to work with exception, java software people has provided the following keywords :

1) try block
2) catch block
3) finally block (try with resources [JDK 1.7V])
4) throw
5) throws

try block :
* try block, **we cannot write independently. It must be followed by either catch block OR finally block OR both.**

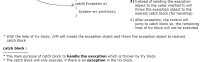


* In between try-catch, the cannot write any kind of statement.
* Whenever **we have error suspecting OR Risky statement** then we should write the statement inside the try block.
Example :

```
try
{
    //Error suspecting statement
                OR
    //Risky Statement

}catch(Exception e)
{
    IO.println(e);
}
```

```
try
{
    int i = 20;
    int j = 30;
    int k = i + j;
    IO.println(k);
}
catch(Exception e)
{
}
```

Flow of try-catch block :




```
try
{
    int i = 20;
    int j = 0;
    int k = i / j;
    IO.println(k);
}
catch(Exception e)
{
    System.err.println(e);
}
```

Work of try block :

1) Check the code line by line
2) If any exception is encountered then JVM will create the exception object and raises the appropriate catch block to handle it.
3) Instead of sending the exception object to the default exception handler JVM throw the exception object to the nearest catch block (for handling).

Work of catch block :
1) Jump to catch block as, the remaining lines of try block will not be executed

* With the help of try block, JVM will create the exception object and throw the exception object to nearest catch block.

catch block :
* The main purpose of catch block is **handle the exception** which is thrown by try block.
* Without catch block, we cannot handle the exception thrown by try block.
* After handling the exception the program will be executed normally.

//Program on try catch :

```
package com.rev1.basic;

import java.util.Scanner;

public class TryDemo
{
    public static void MAIN(String[] args)
    {
        IO.println("Main method started");
        Scanner sc = new Scanner(System.Kb);
        try
        {
            IO.print("Enter the value of a :");
            int a = Integer.parseInt(sc.nextLine());
            IO.print("Enter the value of b :");
            int b = Integer.parseInt(sc.nextLine());
            int r1=a/b;
            IO.println("Result is :"+r1);
            IO.println("End of try");

        }catch(Exception e)
        {
            IO.println("Inside Catch");
            System.err.println(e);
        }
        sc.close();
        IO.println("Main method ended");
    }

}
```

In the above program, if we put the value of y as 0 but still program will be executed **normally** because we have used try-catch so it is a normal termination even we have an exception in the program.

---

import java.util.*;

class ExceptionDemo

```
    public static void main(String[] args)
    {
        try
        {
            //new a = 10/0;

            //throw new ArithmeticException("We are dividing by zero");

        }
        catch (Exception e)
        {
            IO.println("Inside Catch");
            IO.println(e);
        }
    }

}
```

* First time whenever program is run then JVM is internally creating the exception object with the help **of JVM** and throwing the exception object to the nearest catch block.
* We can also create the exception object manually and our throw the exception object to the catch block.
* Main throw keyword can't write any kind of statement, it will become a compilation error.